

Preliminary Results

Ryan Waterman

Table of Contents

Data Preprocessing..... 3

Baseline and Advanced Models 4

Model Analysis and Comparison 7

Recommendations for Phase 3 8

Appendix 9

Data Preprocessing

The data preprocessing process conducted during the initial experimentation can be broken into three subsections: data cleaning and preparation, vectorization and embedding, and train-test data split. Each of these subsections are explained, in detail, below.

Data cleaning and preparation were performed through an ensemble of text processing methods. Due to the unstructured nature of text data, the text processing pipeline's primary purpose was to standardize the text format and remove erroneous data (HTML entities and twitter handles) and common, low signal words (stop words such as "the" and "of"). The exact pipeline is listed, in detail, below:

1. Lowercasing – All text was converted to lowercase to increase uniformity.
2. Demojitization – All emojis were converted to a text representation of the emoji. This was performed to retain the emotion associated with the emojis used, as this is a sentiment analysis task.
3. Twitter Handle Removal – The twitter handles do not add sentiment signal to the tweet content and may even distort the true sentiment of the tweet (i.e. twitter handles such as @happydog123 or @sadcat456).
4. Removal of HTML entities and URLs – This data is purely noise compared to traditional English text.
5. Removal of Special Characters – Leave only text and numbers in the dataset.
6. Tokenization – Utilize the NLTK package to word tokenize the data.
7. Stop Word Removal – Utilize NLTK's stop word list to remove all stop words from the data.
8. Stemming and Lemmatization (Optional) – The preprocessing function also allows for optional stemming or lemmatization. The application of this technique will be explored further in the Baseline and Advanced Model sections.

The model architecture influences the mechanism through which vectorization and embedding were performed. The base model was a Support Vector Machine (SVM) that utilized TF-IDF for vectorization. TF-IDF was selected as the vectorization method as it performs better with sparse data, amplifying the signal of less common words in the global dataset. The advanced model, which was fine tuned from a pretrained BERT base model, utilized the tokenizer, encoder, and decoder from the base model. This simplified the approach, as the data did not require manual tokenization or embedding. Two configurations of preprocessing were used for the advanced model to investigate the impact on performance. The first approach was to pass the data directly to the tokenizer without any preprocessing. The second approach was to apply the preprocessing pipeline, sans tokenization. The impact of these configurations can be found in the Model Comparison section.

Lastly, the train-test split selected for the entirety of the analysis was 70% train, 30% test with a random seed of 42. Validation data was randomly selected from the original data set, then compared against the training and test data to remove any overlap. This resulted in a validation sample size of 9925. Note that preprocessing was applied after selecting the validation set to ensure testing between preprocessing and non-preprocessing was consistent.

Baseline and Advanced Models

As previously mentioned in the Data Preprocessing section, two model architectures were tested in this phase; SVM and BERT, specifically “google-bert/bert-base-uncased” from HuggingFace. The SVM classifier was treated as the baseline model to understand how traditional ML performs on a sentiment analysis task. The advanced model was fine tuned in two configurations; first with a frozen BERT base model (only fine tuning the classifier layer), then fine tuning the entire model. Additional testing was done to determine effect of data preprocessing, as mentioned in the Data Preprocessing. The initial performance evaluation was completed with 3 training epochs on each BERT configuration. The confusion matrices and ROC curves for each tested model can be found, below:

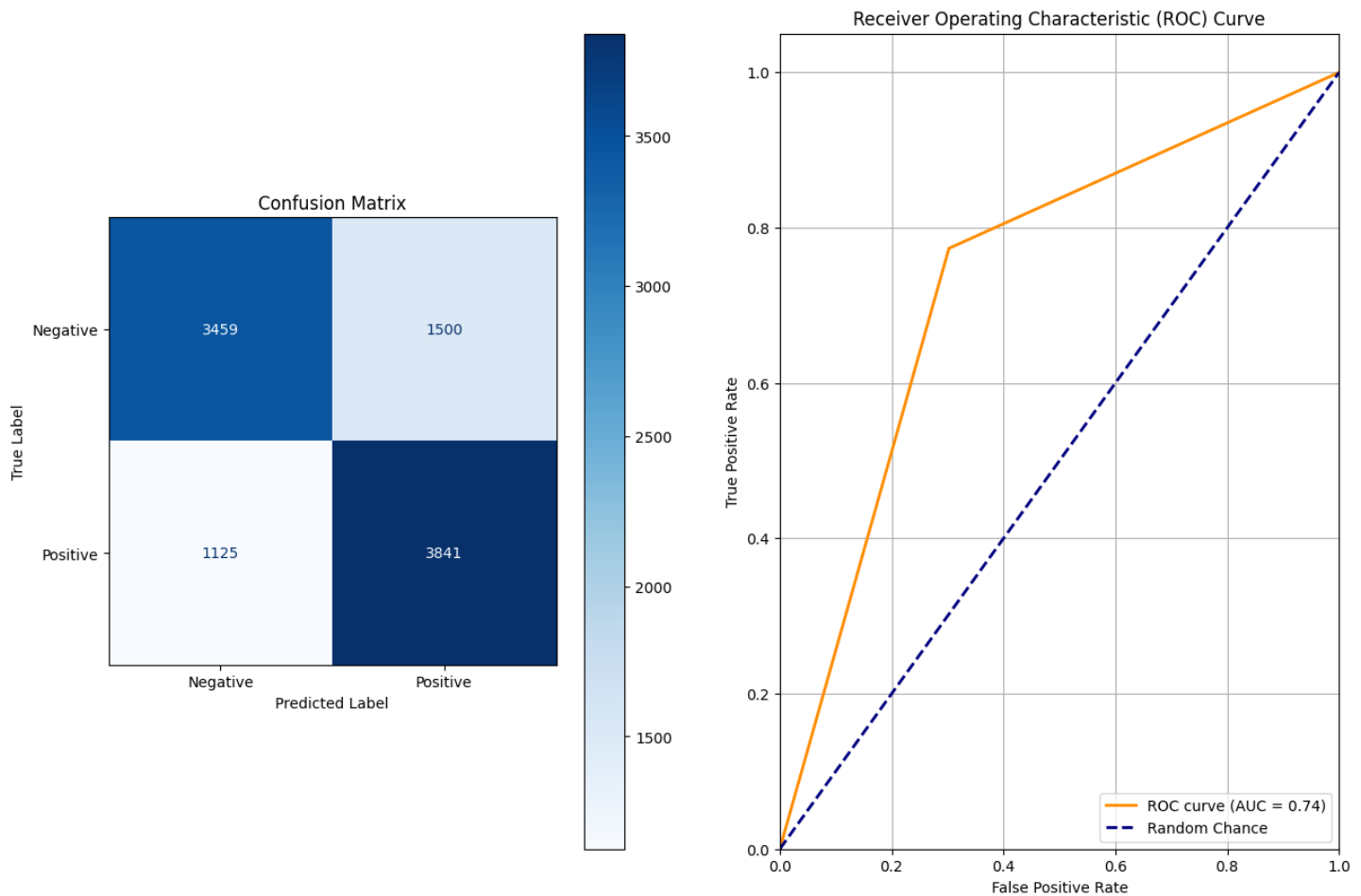


Image 1: CM and ROC for SVM Model

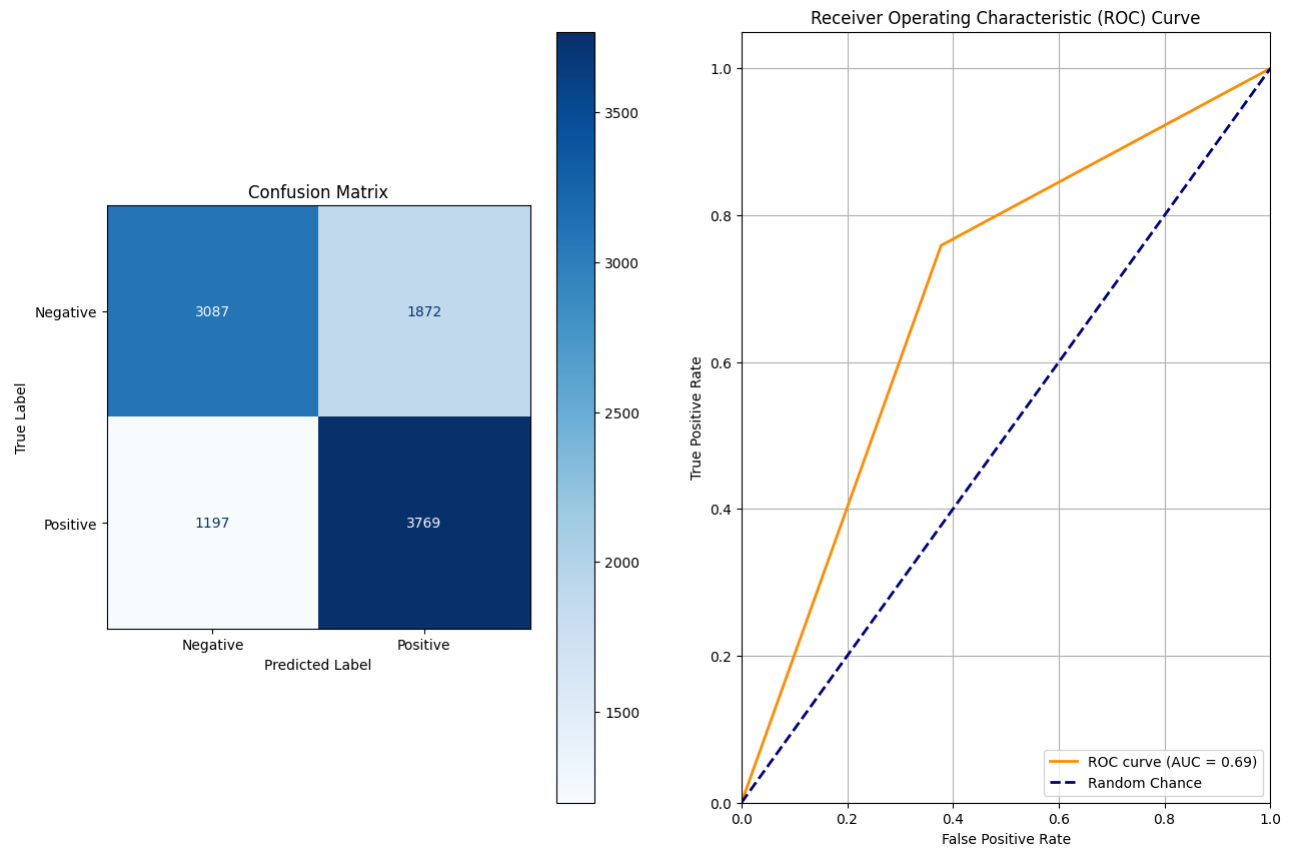


Image 2: CM and ROC for BERT with Frozen Base – No Text Preprocessing

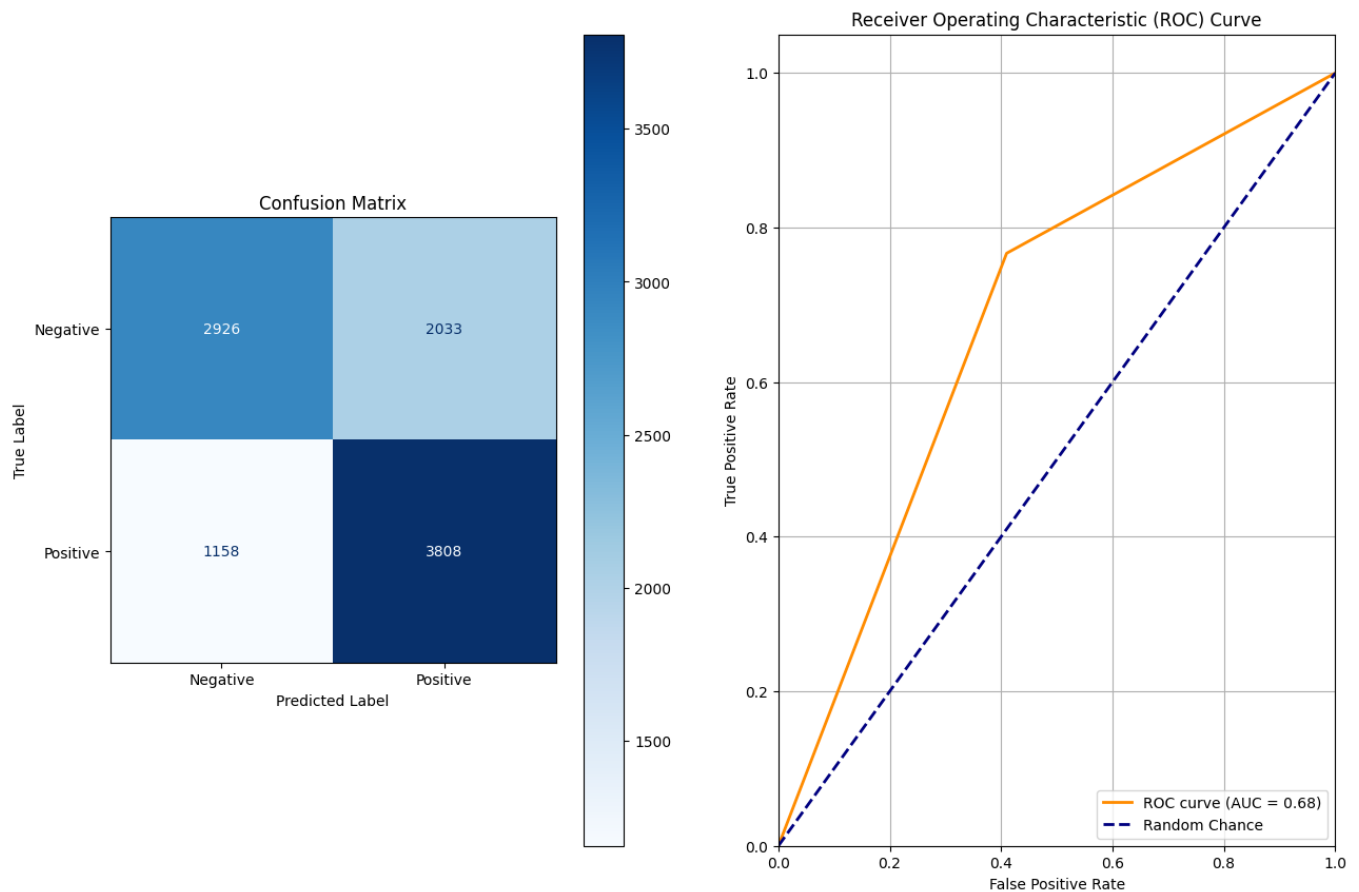


Image 3: CM and ROC for BERT with Frozen Base – Preprocessing

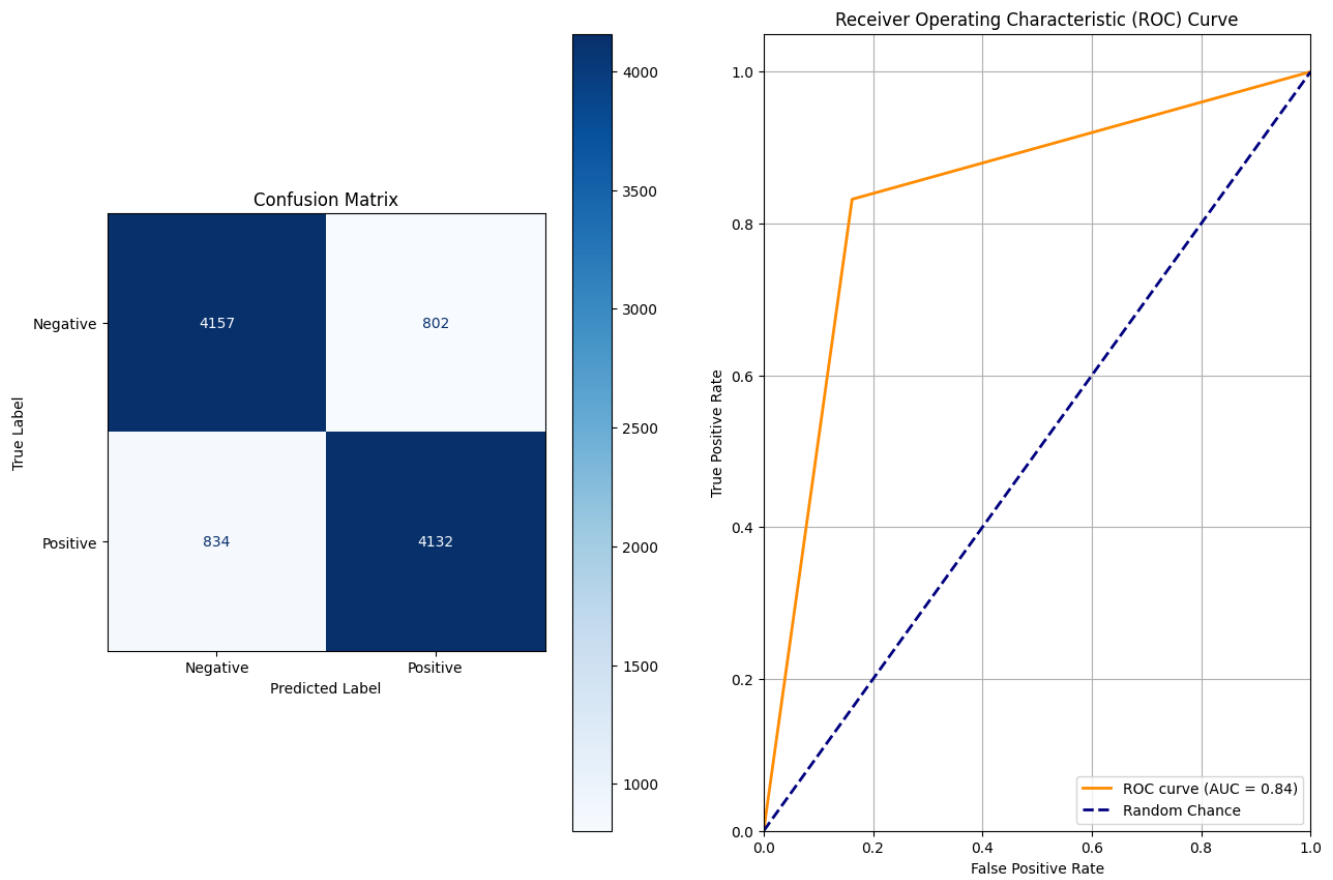


Image 4: CM and ROC for Full BERT Model – No Text Preprocessing

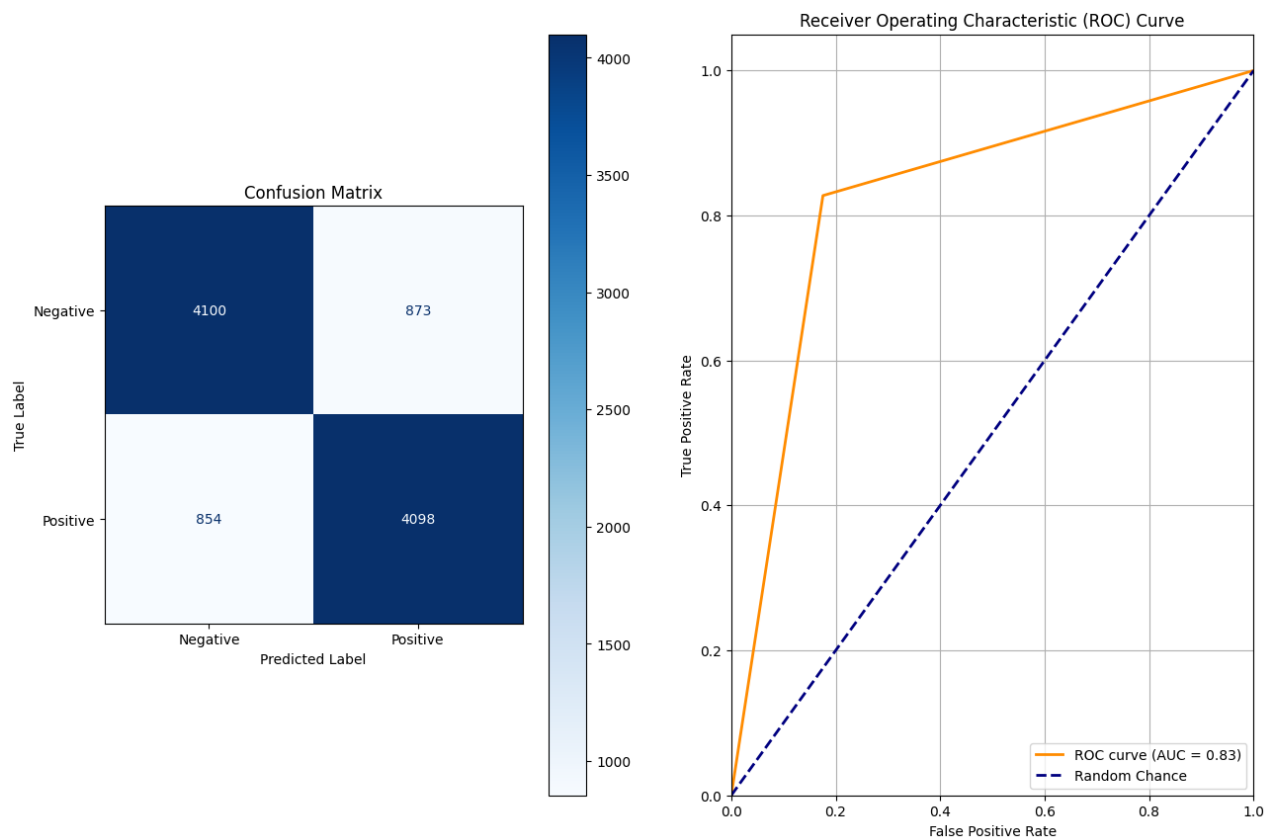


Image 5: CM and ROC for Full BERT Model – Preprocessing

Critical performance metrics for the baseline models are found in Table 1, below:

Model Type	Accuracy	Recall	Precision	F1-Score
SVM	74.05%	74.92%	73.89%	0.74
BERT Frozen without Preprocessing	69.08%	75.90%	66.81%	0.71
BERT Frozen with Preprocessing	67.85%	76.68%	65.19%	0.70
BERT Full Model without Preprocessing	83.52%	83.21%	83.74%	0.83
BERT Full Model with Preprocessing	82.60%	82.75%	82.44%	0.83

Table 1: Baseline Model Performance Metrics

By all metrics, the fine-tuned full BERT model was the best performing model. An in-depth comparison of model performance can be found in the Model Analysis and Comparison section, where the impact of additional training epochs was tested in conjunction with layer freezing and text preprocessing.

Model Analysis and Comparison

After training each of the base BERT configurations, the number of training epochs targeted for further exploration. During training, there was consistent reduction in the loss associated with each model when the third epoch was reached. The cause of this loss reduction was not clear during training, so further evaluation was pursued. The table, below, describes the results for each additional training configuration (CM and ROC Curves found in the appendix).

Model Type	Epoch Count	Accuracy	Recall	Precision	F1-Score
BERT Frozen without Preprocessing	25	73.55%	77.34%	71.91%	0.75
BERT Frozen with Preprocessing	25	72.30%	79.94%	69.27%	0.74
BERT Full Model without Preprocessing	5	82.12%	80.69%	83.08%	0.82
BERT Full Model with Preprocessing	5	81.62%	80.11%	82.54%	0.81

Table 2: Model Performance Metrics for Higher Epoch Intervals

Interestingly, the frozen BERT models both performed better with the higher epoch count, presumably because the classification layer was the only layer undergoing weight and bias updates. The classification layer is a dense layer made up of 768 inputs and 2 outputs, which has no pretrained representation of the input data. Note that the base BERT model is not intended for classification use, but instead it is a generalized language model with internal representations of English text. This text is most likely not representative of twitter data, so the frozen model struggles to train the classifier layer sufficiently in just 3 epochs. One can surmise that this is the root cause behind the degradation in performance with preprocessed data; the model is not trained on the

resulting, fragmented data. The full BERT model, contrary to the frozen model, performs worse with 2 additional training epochs. One outcome that can arise from fine-tuning the BERT model is catastrophic forgetting, which occurs when the base model's internal representation of the language begins to be obscured by the new training data, causing the model to "forget" language representations, which appears to be occurring here.

Recommendations for Phase 3

The fine-tuned full BERT model was the most successful model for sentiment prediction during this phase of the project. This model architecture will be used in phase 3, where the intent is to perform testing on many more of the hyperparameters. For example, batch size, max input length, freezing specific combinations of layers, learning rate, and the tokenizer can all be modified in tandem to optimize the model for sentiment classification. An 83% accuracy baseline is a strong start for continuous testing and optimization, but a significant improvement must be made for this model to be production ready.

Appendix

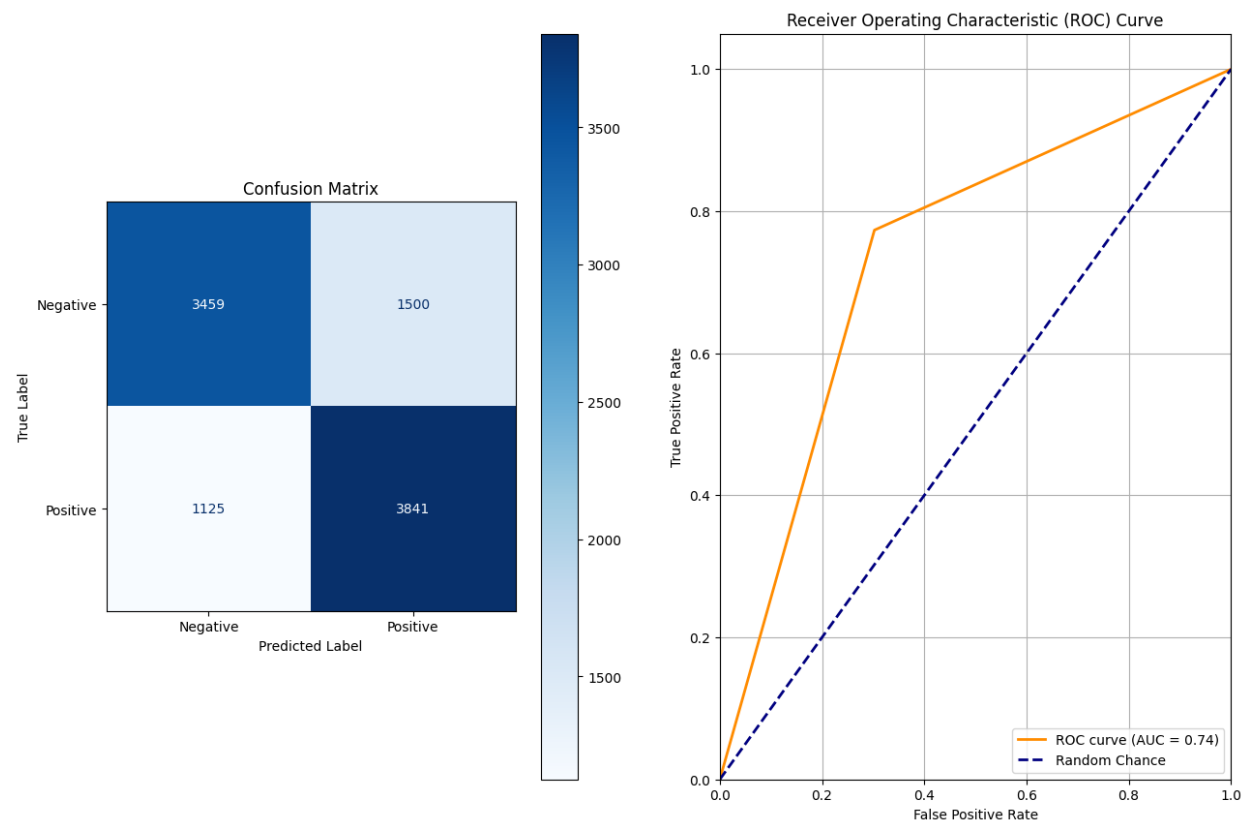


Image 6: CM and ROC for BERT with Frozen Base – No Text Preprocessing – 25 Epochs

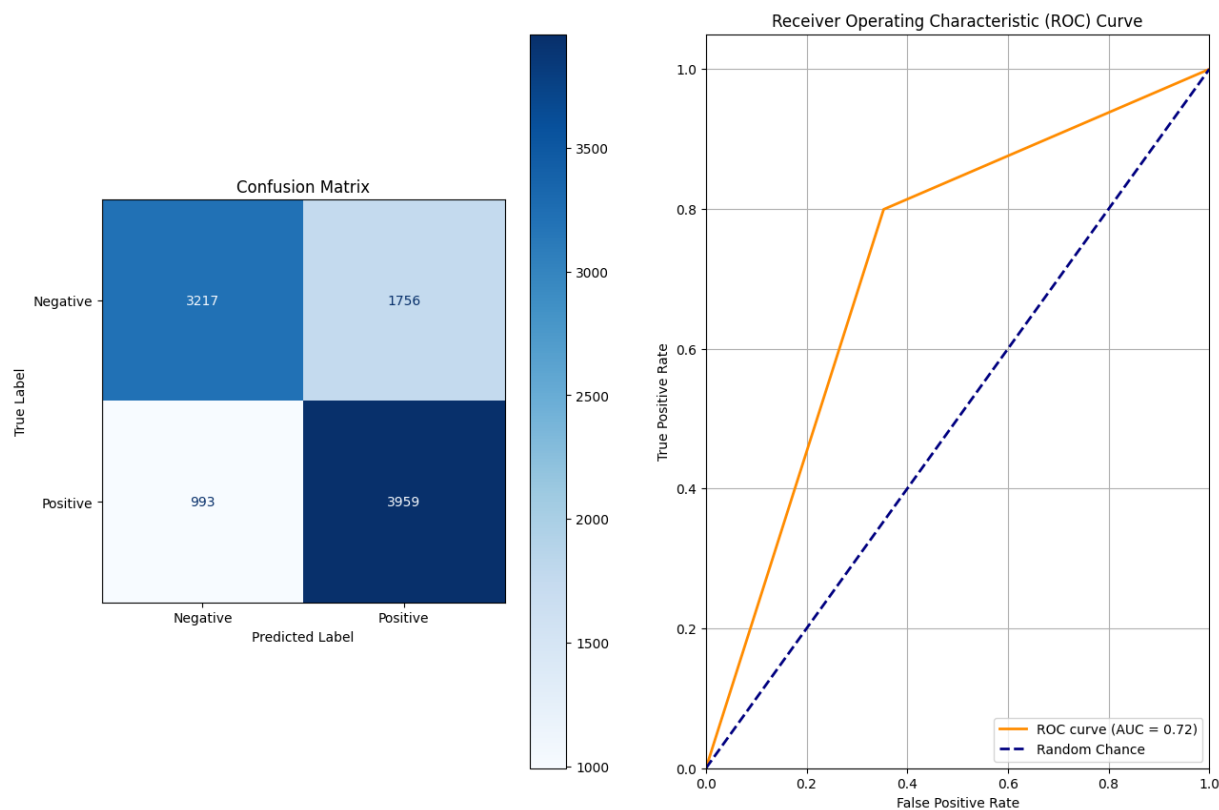


Image 7: CM and ROC for BERT with Frozen Base – Preprocessing – 25 Epochs

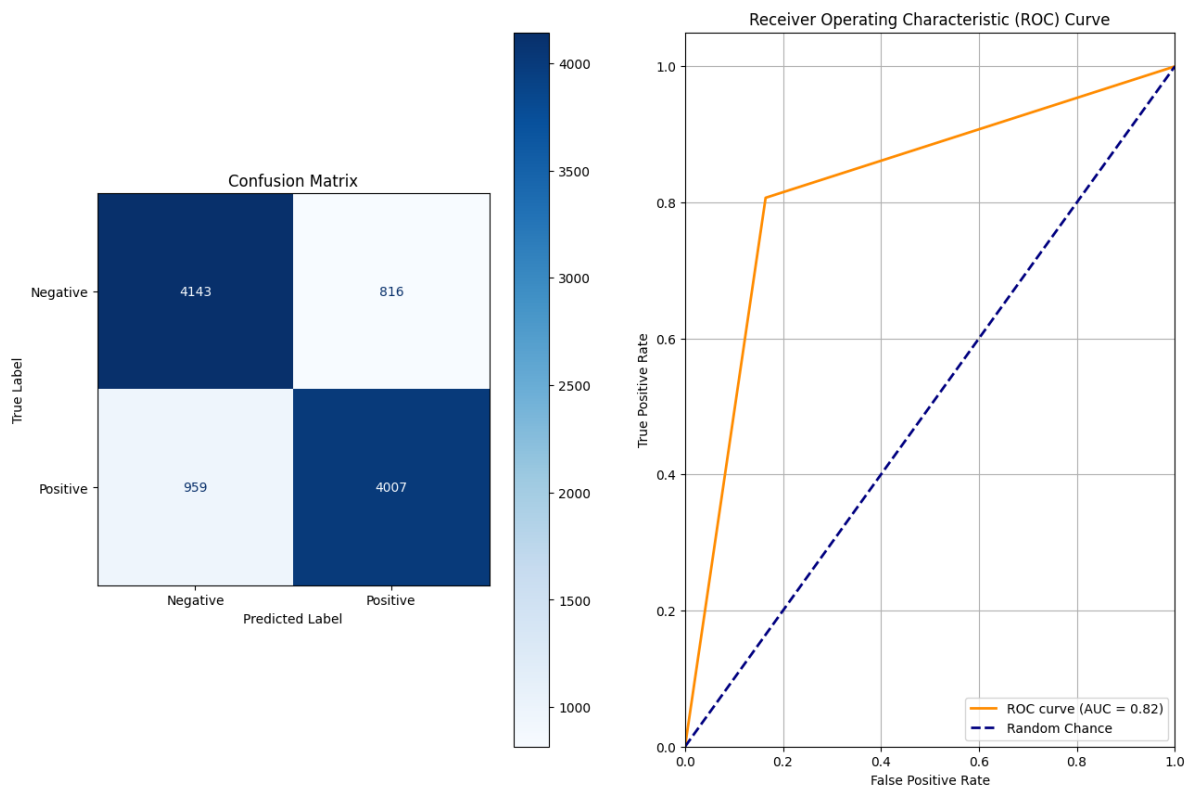


Image 8: CM and ROC for Full BERT – No Text Preprocessing – 5 Epochs

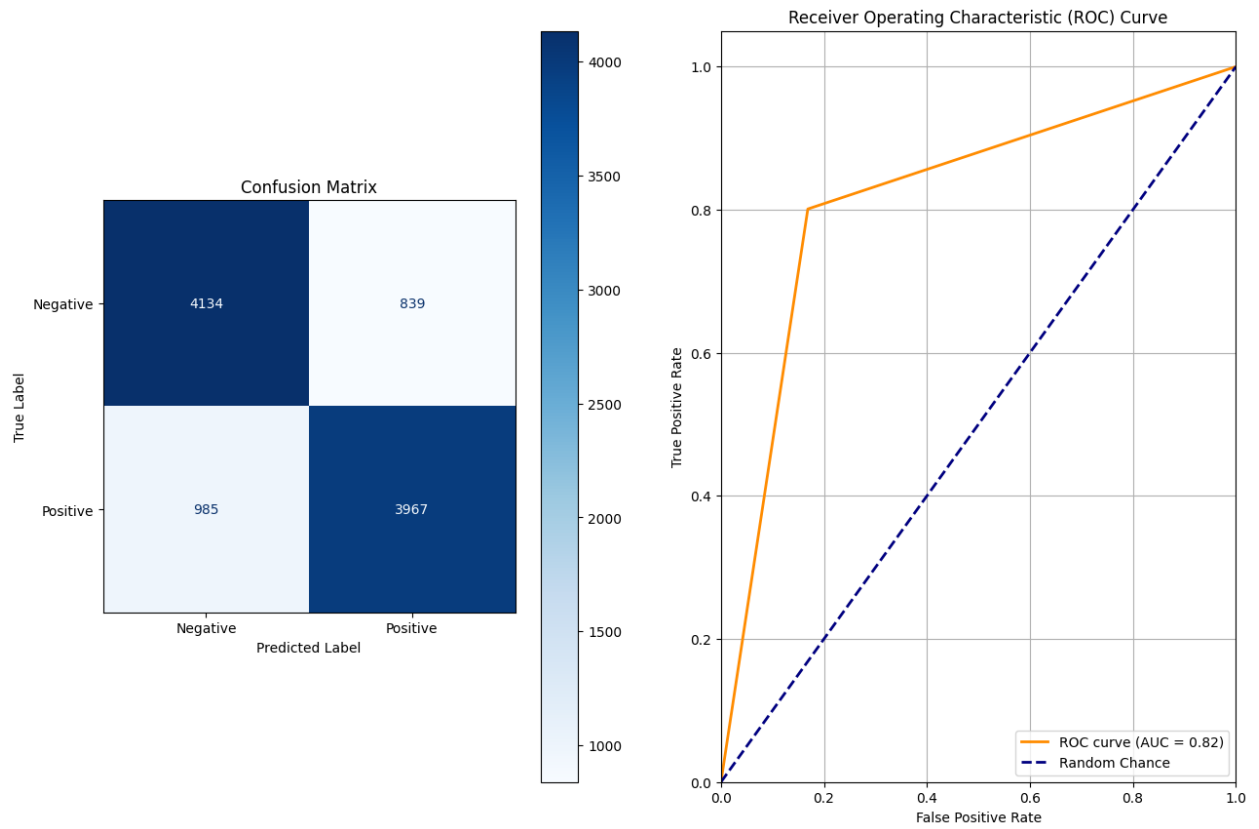


Image 8: CM and ROC for Full BERT – Text Preprocessing – 5 Epochs